

COS 433: Cryptography

Reminders:

- Problem Set 0 (survey) due on Wednesday. Submit on Gradescope.
- Problem Set 1 to be released on Wed, due two weeks after
 - 100 points = full score (>> 100 available points)
- Office Hours:

Mondays 5-6pm, CS 003

Thursdays 12pm-1pm, CS 312

Secret-Key Encryption

(T, ϵ) Indistinguishability

Definition: Two random variables $X \approx_{T, \epsilon} Y$ are (T, ϵ) -computationally indistinguishable if for all “distinguishers” $D: \mathcal{X} \rightarrow \{0,1\}$ computable in time T ,

$$|\mathbf{E}[D(X)] - \mathbf{E}[D(Y)]| \leq \epsilon$$

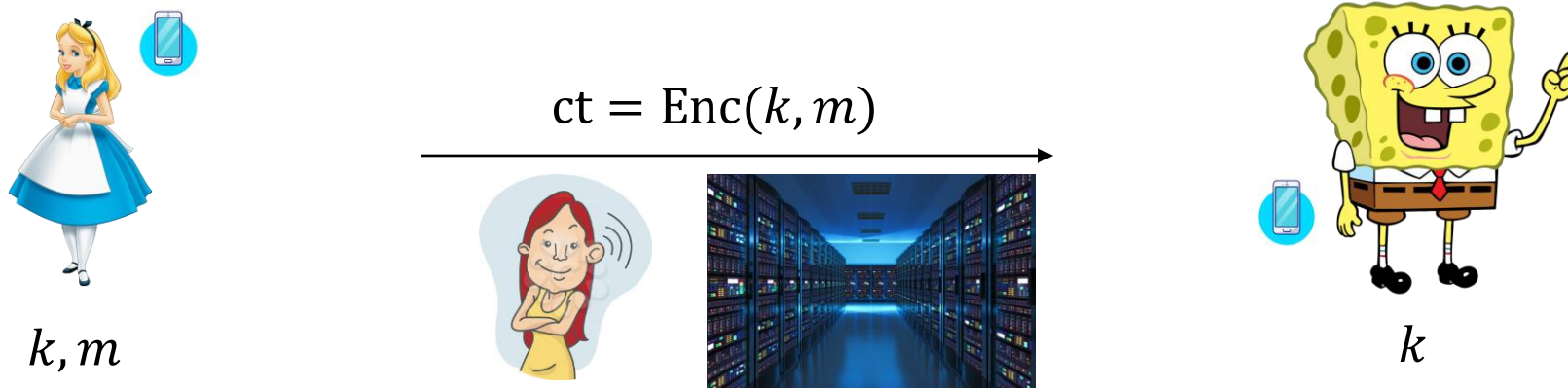
Definition: a one-time secret-key encryption scheme (Enc, Dec) is (T, ϵ) -computationally secure if for all message pairs (m_0, m_1)

$$\text{Enc}(k, m_0) \approx_{T, \epsilon} \text{Enc}(k, m_1)$$

(T, ϵ) Indistinguishability

General principle:

Assume that the adversary is more powerful than the honest parties.



- Looking for (T, ϵ) -secure encryption schemes where the honest parties run in much less than T time.
- Real-world tradeoffs between efficiency and security.

(T, ϵ) Indistinguishability

Questions:

1. Is the modeling assumption valid? Depends on (T, ϵ) .
2. Do (T, ϵ) -secure encryption schemes exist?
 - The modern world depends on it, but we don't know for sure!
3. How do we build such a scheme?

(T, ϵ) Indistinguishability

Questions:

1. Is the modeling assumption valid? Depends on (T, ϵ) .

Time Estimation (don't try this at home)

Speed of a modern laptop: 2^{37} floating point operations per second.

Speed of a **modern supercomputer**: 2^{60} floating point operations per second.

1000 years = 2^{35} seconds.

Age of the universe: $2^{58.5}$ seconds.

Can execute **2^{120} floating point operations** running a modern supercomputer for the current lifetime of the universe.

In the real world, usually set $T/\epsilon = 2^{128}, 2^{256}, 2^{512}$

Time Estimation (don't try this at home)

Concerns not addressed in this calculation:

- What about parallelization?
- Validity of the Boolean circuit computational model (Church-Turing thesis)
 - Now believed by many to be **false** (e.g., “quantum computers”)
- “Future-proofing”

In the real world, usually set $T/\epsilon = 2^{128}, 2^{256}, 2^{512}$

(T, ϵ) Indistinguishability

Questions:

2. Do (T, ϵ) -secure encryption schemes exist?
 - The modern world depends on it, but we don't know for sure!

Do (T, ϵ) -secure encryption schemes exist?

Does there exist a secret-key encryption scheme (Enc, Dec) with:

- $\mathcal{K} = \{0,1\}^\lambda$
- $\mathcal{M} = \{0,1\}^n$
- $n \geq \lambda + 1$
- (T, ϵ) security for reasonable T, ϵ ?

Intuition: there's always a **brute force attack**

- Enumerate all keys (or only T random ones)
- Decrypt the ciphertext
- Check if m_0 or m_1

Do (T, ϵ) -secure encryption schemes exist?

Does there exist a secret-key encryption scheme (Enc, Dec) with:

- $\mathcal{K} = \{0,1\}^\lambda$
- $\mathcal{M} = \{0,1\}^n$
- $n \geq \lambda + 1$
- (T, ϵ) security for reasonable T, ϵ ?

Conclusion: $\frac{T}{\epsilon} \leq 2^\lambda$

Is this okay? Yes!

Do (T, ϵ) -secure encryption schemes exist?

Does there exist a secret-key encryption scheme (Enc, Dec) with:

- $\mathcal{K} = \{0,1\}^\lambda$
- $\mathcal{M} = \{0,1\}^n$
- $n \geq \lambda + 1$
- (T, ϵ) security for reasonable T, ϵ ?

Conclusion: $\frac{T}{\epsilon} \leq 2^\lambda$

Real world schemes: best known attack on **AES-128** takes 2^{126} time

Do (T, ϵ) -secure encryption schemes exist?

Does there exist a secret-key encryption scheme (Enc, Dec) with:

- $\mathcal{K} = \{0,1\}^\lambda$
- $\mathcal{M} = \{0,1\}^n$
- $n \geq \lambda + 1$
- (T, ϵ) security for reasonable T, ϵ ?

Intuition: seems hard to generically shortcut the brute force attack.

Do (T, ϵ) -secure encryption schemes exist?

Does there exist a secret-key encryption scheme (Enc, Dec) with:

- $\mathcal{K} = \{0,1\}^\lambda$
- $\mathcal{M} = \{0,1\}^n$
- $n \geq \lambda + 1$
- (T, ϵ) security for reasonable T, ϵ ?

Curse of complexity theory

It's really hard to **prove** that problems take a long time to solve.

“Circuit lower bounds.”



(Asymptotic) Computational Security

For the remainder of this course: “efficient” means **polynomial running time** as a function of the **security parameter $\lambda \rightarrow \infty$** .

- By default: $\mathcal{K} = \{0,1\}^\lambda$
- $\mathcal{M} = \{0,1\}^n, n = \text{poly}(\lambda) = \lambda^C$ for some constant $C > 0$
- The honest algorithms Enc, Dec run in time $\text{poly}(\lambda, n) = \text{poly}(\lambda)$
- Adversary is a collection of algorithms $A^{(\lambda)}: \{0,1\}^* \rightarrow \{0,1\}$ for every security parameter

Computational Indistinguishability

Definition: Two **ensembles of random variables** $(X^{(\lambda)})_{\lambda \in \mathbb{N}} \approx_c (Y^{(\lambda)})_{\lambda \in \mathbb{N}}$ are called **computationally indistinguishable** if for every constant $C > 0$,

$$X^{(\lambda)} \approx_{\lambda^C, o(\lambda^{-C})} Y^{(\lambda)}$$

For all $T = \text{poly}(\lambda)$, and for all time T distinguisher families $(D^{(\lambda)}: \{0,1\}^* \rightarrow \{0,1\})$, **there exists a negligible function $\epsilon = \text{negl}(\lambda)$** such that

$$|\mathbf{E}[D^{(\lambda)}(X^{(\lambda)})] - \mathbf{E}[D^{(\lambda)}(Y^{(\lambda)})]| \leq \epsilon(\lambda)$$

A function $\epsilon(\lambda)$ is **negligible** if for all $C > 0$, $\epsilon(\lambda) = o\left(\frac{1}{\lambda^C}\right)$. ($\epsilon(\lambda) = 2^{-\lambda}$, $\epsilon(\lambda) = \lambda^{-\log(\lambda)}$)

Computational Indistinguishability

Negligible functions and computational indistinguishability behave nicely under simple transformations.

Lemma: If $\epsilon_1(\lambda)$ and $\epsilon_2(\lambda)$ are two negligible functions, then so is $\epsilon_1(\lambda) + \epsilon_2(\lambda)$

Proof: For all $C > 0$,

$$\epsilon_1(\lambda) + \epsilon_2(\lambda) = O(\lambda^{-C}) + O(\lambda^{-C}) = O(\lambda^{-C})$$

Computational Indistinguishability

Negligible functions and computational indistinguishability behave nicely under simple transformations.

Lemma: If $X \approx_c Y$ and $Y \approx_c Z$ then $X \approx_c Z$

Proof: Let D be a $\text{poly}(\lambda)$ time distinguisher. Let

$$\epsilon_1(\lambda) = |\mathbf{E}[D^{(\lambda)}(X^{(\lambda)})] - \mathbf{E}[D^{(\lambda)}(Y^{(\lambda)})]|$$

$$\epsilon_2(\lambda) = |\mathbf{E}[D^{(\lambda)}(Y^{(\lambda)})] - \mathbf{E}[D^{(\lambda)}(Z^{(\lambda)})]|$$

Then

$$|\mathbf{E}[D^{(\lambda)}(X^{(\lambda)})] - \mathbf{E}[D^{(\lambda)}(Z^{(\lambda)})]| \leq \epsilon_1(\lambda) + \epsilon_2(\lambda) = \text{negl}(\lambda)$$

Secret-Key Encryption: Final Definition



k, m

$$ct = \text{Enc}(k, m)$$



k

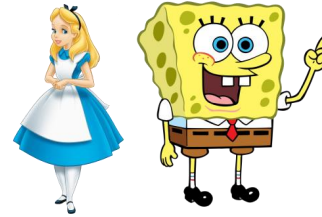
$$m' = \text{Dec}(k, ct)$$

Definition: A **computational** secret-key **one-time** encryption scheme with message length $n = n(\lambda)$ consists of algorithms (Enc, Dec) . It must satisfy the following properties.

- **Efficiency:** given a key $k \in \{0,1\}^\lambda$ and message $m \in \{0,1\}^{n(\lambda)}$ as input, Enc runs in polynomial time. Similar for $\text{Dec}(k, ct)$.
- **Correctness:** $\text{Dec}(k, \text{Enc}(k, m)) = m$ for all k, m .
- **Computational one-message security:** for all (m_0, m_1) , $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$.

Secret-Key Encryption: Final Definition

Philosophy: large gap between work required of



compared to



→ **some large enough** (but not too large) λ should work (real world dependent)

Definition: A computational secret-key one-time encryption scheme with message length $n = n(\lambda)$ consists of algorithms (Enc, Dec). It must satisfy the following properties.

- **Efficiency:** given a key $k \in \{0,1\}^\lambda$ and message $m \in \{0,1\}^{n(\lambda)}$ as input, Enc runs in polynomial time. Similar for Dec(k , ct).
- **Correctness:** $\text{Dec}(k, \text{Enc}(k, m)) = m$ for all k, m .
- **Computational security:** for all $(m_0 = \{m_0^{(\lambda)}\}, m_1 = \{m_1^{(\lambda)}\})$, $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$.

Secret-Key Encryption

So, do such schemes exist??

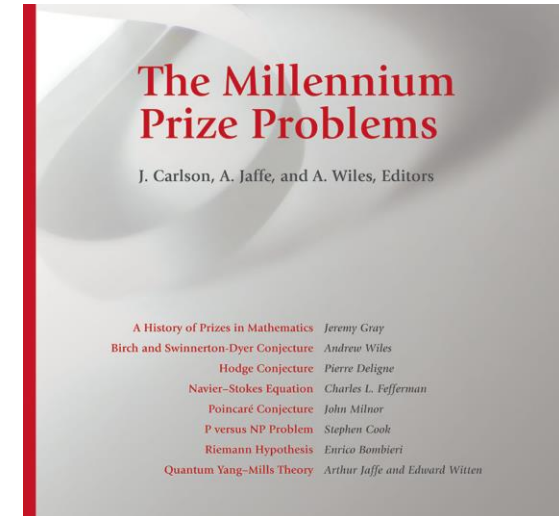
Theorem: if $P = NP$, then secret-key encryption schemes with $n(\lambda) > \lambda$ **do not exist**.

Proof sketch: if $P = NP$, the brute force attack can be done in polynomial time.



Thus, all of our constructions will be either:

- **Heuristic** (we don't like this), or
- Conditional on **conjectures**



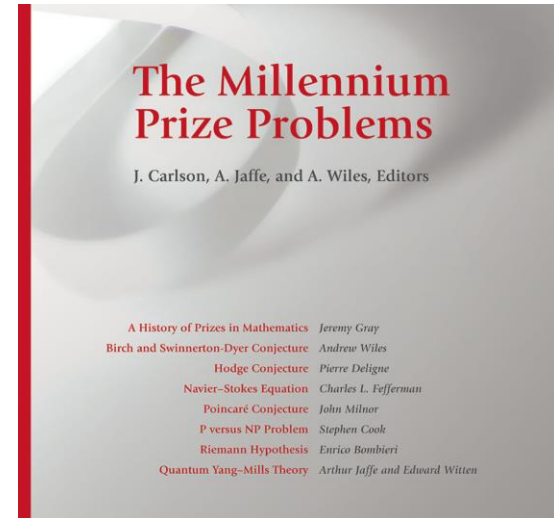
Secret-Key Encryption

Goals:

1. Understand what tools are necessary/sufficient to build crypto
2. **Minimize** the strength of the **conjectures** we assume
3. Produce **evidence** for the conjectures
4. Justify **paradigms** for building crypto

Thus, all of our constructions will be either:

- **Heuristic** (we don't like this), or
- Conditional on **conjectures**



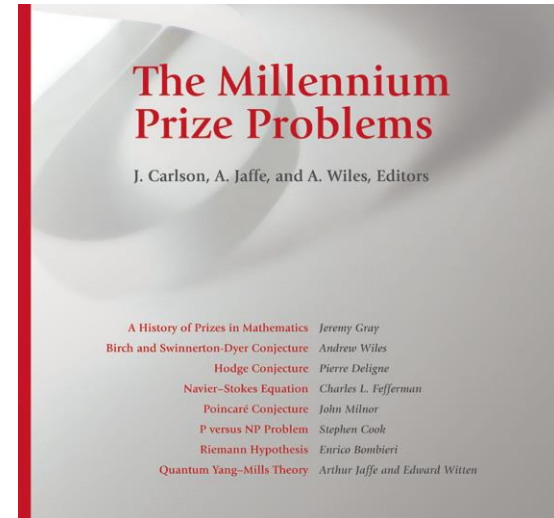
Secret-Key Encryption

Open Problem:

Can we prove that if $P \neq NP$, then encryption (for long messages) is possible?

Thus, all of our constructions will be either:

- **Heuristic** (we don't like this), or
- Conditional on **conjectures**

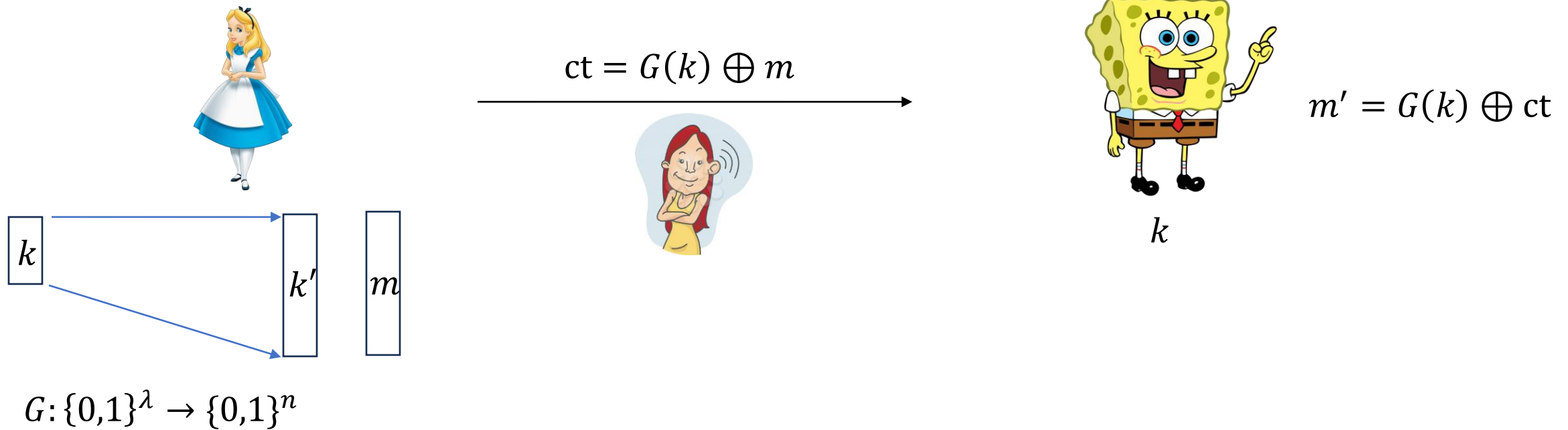


Computational Indistinguishability

Questions:

3. How do we build such a scheme?

A construction!

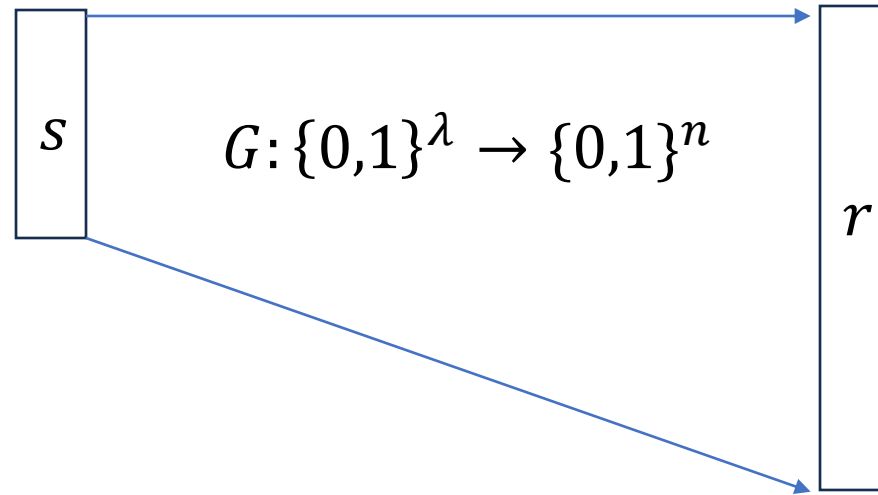


Simple proposal: for some efficiently computable function G , define

$$\text{Enc}(k, m) = G(k) \oplus m$$

$$\text{Dec}(k, ct) = G(k) \oplus ct$$

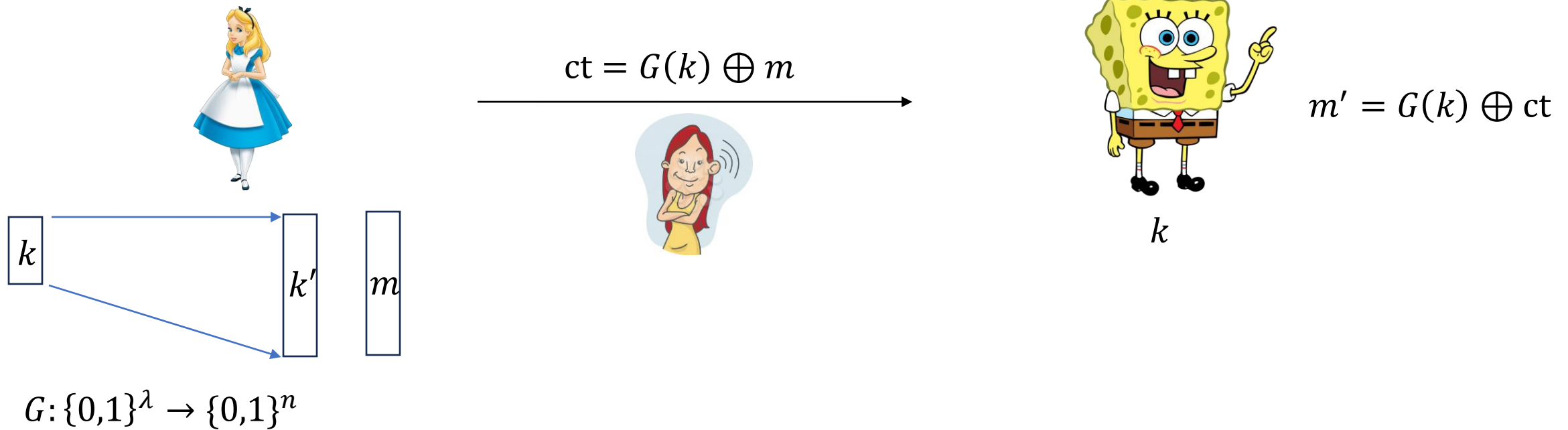
(Cryptographic) Pseudorandom Generators



Definition: An **efficiently computable** function family $G = \{G^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}\}_\lambda$ is a **pseudorandom generator (PRG)** if it satisfies two additional properties:

- Stretch: $n(\lambda) > \lambda$ for all λ
- Pseudorandomness: $G(U_\lambda) \approx_c U_n$ (uniform n bit string)

A construction!



Theorem: if G is a PRG, then this is a secure one-time encryption scheme!

Security “Proof by Reduction”



k, m

$$ct = G(k) \oplus m$$



k

$$m' = G(k) \oplus ct$$

Proof:

- If G is efficiently computable, then so are **Enc**, **Dec**
- Correctness: $G(k) \oplus (G(k) \oplus m) = m$

Security “Proof by Reduction”



k, m

$$ct = G(k) \oplus m$$



k

$$m' = G(k) \oplus ct$$

Proof: Want to show that if G is a PRG, then $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$

Hybrid argument

$$G(\textcolor{red}{k}) \oplus \textcolor{blue}{m}_0 \approx_c U_n \approx_c G(\textcolor{red}{k}) \oplus \textcolor{blue}{m}_1$$

Security “Proof by Reduction”

Want to show that if G is a PRG, then $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$

Claim: for all message tuples $m = (m^{(\lambda)})_\lambda$, $G(k) \oplus m \approx_c U_n$

Proof: suppose D is an efficient distinguisher violating the claim:

$$|\mathbf{E}[D(G(k) \oplus m)] - \mathbf{E}[D(r)]| = \epsilon(\lambda)$$

$\epsilon(\lambda)$ is non-negligible: $\exists c: \epsilon(\lambda) \geq \lambda^{-c}$ for infinitely many λ

Security “Proof by Reduction”

Want to show that if G is a PRG, then $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$

Claim: for all message tuples $m = (m^{(\lambda)})_\lambda$, $G(k) \oplus m \approx_c U_n$

Proof: suppose D is an efficient distinguisher violating the claim:

$$|\mathbf{E}[D(G(k) \oplus m)] - \mathbf{E}[D(r)]| = \epsilon(\lambda)$$

We'll show how to break G . (Proof by contradiction)

Security “Proof by Reduction”

Want to show that if G is a PRG, then $\text{Enc}(k, m_0) \approx_c \text{Enc}(k, m_1)$

Claim: for all message tuples $m = (m^{(\lambda)})_\lambda$, $G(k) \oplus m \approx_c U_n$

Proof: suppose D is an efficient distinguisher violating the claim:

$$|\mathbf{E}[D(G(k) \oplus m)] - \mathbf{E}[D(r)]| = \epsilon(\lambda)$$

Define a new distinguisher:

$$D'(x) = D(x \oplus m)$$

Observe: $|\mathbf{E}[D'(G(k))] - \mathbf{E}[D'(r)]| = \epsilon(\lambda) \quad \implies \Longleftarrow$